

РАЗРАБОТКА ИИ / БОЛЬШИЕ ЯЗЫКОВЫЕ МОДЕЛИ / БЕЗОПАСНОСТЬ

Более чистые данные для обучения ИИ, меньше ошибок: что такое SonarSweep от Sonar

SonarSweep фильтрует некачественные обучающие данные для ИИ, сокращая количество ошибок и уязвимостей в сгенерированном коде на 41%. Узнайте, как это работает.

11 июня 2026 года, 8:00 [Джо Тайлер](#)



Убайд Э. Аляфизи для Unsplash+

[Sonar](#) спонсировала этот пост. Insight Partners является инвестором Sonar и TNS.

Большие языковые модели быстро превратились из новинки в повседневную инфраструктуру разработки программного обеспечения. Мы больше не используем ИИ только для того, чтобы отвечать на отдельные вопросы в

скоростью, на которую традиционные процессы проверки не рассчитаны.

Доказано, что модели могут генерировать код. Вопрос в том, могут ли команды доверять тому, что они создают. Большая часть проблем возникает на более ранних этапах, в самих **данных, используемых для обучения моделей** — это яркий пример принципа «Мусор на входе — мусор на выходе». Исследовательская группа по искусственному интеллекту в Sonar тщательно изучила этот феномен и разработала технологию под названием **SonarSweep**, призванную решить эту проблему.

Публичные репозитории кода предоставляют моделям огромный выбор языков и фреймворков, но в них также содержатся устаревшие библиотеки, небезопасные шаблоны, хрупкие реализации и плохо поддерживаемые решения. Модели учатся на всем этом. Они не всегда могут отличить примеры, отражающие грамотную разработку, от примеров, которые просто компилируются.

Это важно, потому что функциональный код — это не то же самое, что готовый к использованию код. В корпоративной среде код также должен быть безопасным, удобным для сопровождения и устойчивым к реальным условиям. Если эти качества не соблюдаются в обучающих данных, они не будут соблюдаться и в выходных данных.

Риск использования непроверенного кода

Большие языковые модели — это, по сути, сложные статистические системы. Они не обладают врожденным пониманием того, что такое «хорошая» и «плохая» разработка. Они оптимизируют наиболее вероятное решение на основе предоставленного контекста и закономерностей, выявленных в обучающих данных.



Sonar — это отраслевой стандарт для проверки и автоматизированного анализа кода, которому доверяют 75 % компаний из списка Fortune 100. Платформа SonarQube ежедневно анализирует более 750 миллиардов строк кода, помогая предотвращать сбои, снижать риски, уменьшать

[Learn more](#) →

THE LATEST FROM SONAR

[Announcing Advanced Security for SonarQube Cloud Team plan](#)

24 June 2026

[Now available: SonarQube plugin for Codex](#)

24 June 2026

[Now available: SonarQube plugin for Cursor](#)

24 June 2026

HEAR MORE FROM OUR SPONSOR

svo1975pvn1982@gmail.com[Submit](#)

That means weaknesses in the training corpus can persist in model behavior in ways that are easy to underestimate. Research, including [findings from Anthropic](#) late last year, suggests that even relatively small samples of “poisoned” or low-quality data can be encoded into models’ inner representations and lead to dangerous behaviors.

In a paper I co-authored last year, [“Assessing the Quality and Security of AI-Generated Code: A Quantitative Analysis,”](#) we observed this exact issue. All models considered in our study generated a mixture of simple and sophisticated bugs, vulnerabilities, and code smells (maintainability issues), and there are some interesting trends between models: with certain labs prioritizing security performance while others succeed in writing maintainable code.

TRENDING STORIES

- [1. "Code should be regenerated, not maintained": Codeplain makes the case for spec-driven development](#)
- [2. Gemini CLI vs. Antigravity: What works, not the spec sheet](#)
- [3. Your engineering org needs an AI slop registry](#)

5. Loops are replacing prompts. Verification is about to be your biggest problem.

To highlight the complexity of these issues, consider path traversal vulnerabilities. These typically require taint analysis (following an input source to a sensitive sink) in order to detect them. While an LLM might generate a block of code that performs a specific function correctly, it may fail to account for how unvalidated user input could manipulate file paths or inject malicious commands three functions down the line.

“A model can produce code that looks correct, passes a narrow test, and still introduces issues that increase review time, technical debt and security exposure.”

For enterprise teams, this is the real risk. A model can produce code that looks correct, passes a narrow test, and still introduces issues that increase review time, **technical debt** and security exposure. At scale, those flaws do not stay isolated. They spread through pull requests, internal tools and downstream systems.

Why data quality engineering matters

Organizations are becoming increasingly interested in building and owning their AI. If they want their AI-assisted coding to be context-aware, trustworthy, and cost-effective, they need to treat their code and other data as critical assets. That means applying quality controls before the model ever learns from their datasets.

This is where data quality engineering becomes essential. Rather than accepting public code corpora (and even their own data) as-is, teams can inspect, filter, and improve training data so the model learns from stronger examples.

My team has put this into practice with SonarSweep, an approach that “sweeps” datasets before the model ever sees them. This ensures training data is properly scrutinized, which is crucial for companies seeking to understand and improve their **semantic** development practices. There are four key phases to sweeping datasets:

2. Synthesize examples. Create high-quality training examples for underrepresented tasks and domains. Optionally, an organization may use their codebase to embed specific understanding into the model.
3. Remediate what can be remediated. If insecure or outdated patterns can be automatically corrected, the model can learn from the improved version instead of the flawed one.
4. Curate aggressively. Not every example deserves equal weight in a training set. A quality gate removes low-signal data and maximizes diversity.

The payoff is measurable

This is not just a theoretical argument. Training on “swept” data in our **model release** from the end of last year led to a 41% reduction in the density of security vulnerabilities and a 41% reduction in the density of bugs in the model’s generated output.

That kind of improvement matters because the ROI is not only technical. It is operational. Within an agentic coding session, when the model writes code containing fewer bugs and vulnerabilities, the agent will spend less time and fewer tokens in the loops of the **Agent Centric Development Lifecycle** (AC/DC) — a framework built for how software is developed today. Within the AC/DC method, AI agents generate most of the code, and teams manage the development loops in three core stages: Guide, Verify, and Solve.

Furthermore, training the model on your codebase will reduce token usage and help it get up to speed with your architecture and best practices at the start of a session. Additionally, cleaner, better-structured code reduces token use, as agents will spend less time re-reading files, rebuilding context, and working around unnecessary complexity.

Sonar’s **research** supports that point: Across six matched pairs of codebases and roughly 660 Claude Code task runs, agents working in SonarQube-verified codebases used about 7% fewer input tokens and 8% fewer output tokens, with no meaningful change in task completion.

Generating quality from the ground up

- **next frontier in AI coding** is not simply bigger models or faster generation. It is better foundations. We are reaching the point where the limiting factor is no longer

“The next frontier in AI coding is not simply bigger models or faster generation. It is better foundations.”

To close the trust gap, organizations must build quality into their system from the beginning, including the datasets that shape model behavior before deployment. The teams that do this well will be the ones that get the most value from AI-assisted development, because their agents will be more efficient, spend less time correcting preventable defects, and have more time to put trustworthy software into production.

For engineer leaders and developers, the mandate is clear: development teams need to understand the implications of different agent configurations on their token spend and development output.

TNS



Joe Tyler is a specialist in Large Language Models with a background in natural language processing and real-world AI applications. He is currently an AI Researcher at Sonar based in London, leveraging generative AI to revolutionize code quality and security....

[Read more from Joe Tyler →](#)

TNS owner Insight Partners is an investor in: Anthropic, SonarSource, Sonar.

SHARE THIS STORY



TRENDING STORIES

1. "Code should be regenerated, not maintained": Codeplain makes the case for spec-driven development
2. Gemini CLI vs. Antigravity: What works, not the spec sheet
3. Your engineering org needs an AI slop registry

5. Loops are replacing prompts. Verification is about to be your biggest problem.

INSIGHTS FROM OUR SPONSOR



Sonar is the industry standard for code verification and automated code review, trusted by 75% of the Fortune 100. Its SonarQube platform analyzes over 750 billion lines of code daily, helping to prevent outages, reduce risk, lower technical debt, and ensure compliance.

[Learn More →](#)

Announcing Advanced Security for SonarQube Cloud Team plan

24 June 2026

Now available: SonarQube plugin for Codex

24 June 2026

Now available: SonarQube plugin for Cursor

24 June 2026

What NIST should know when updating the SSDF for AI

18 June 2026

AI is writing more of your Terraform

18 June 2026

Your AI bill is a code quality problem

17 June 2026

most recent TNS articles in your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter

THE NEW STACK

[About / Contact](#)
[Sponsors](#)
[Advertise With Us](#)
[Contributions](#)



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

[Frontend Developer Roadmap](#)
[Backend Developer Roadmap](#)
[Devops Roadmap](#)

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)
[Terms of Use](#)
[Advertising Terms & Conditions](#)
[Privacy Policy](#)
[Cookie Policy](#)